

Dispositivos Conectados

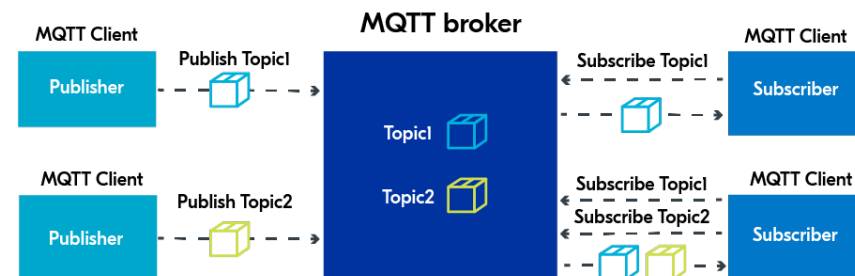
Protocolos IP para IoT: Modelo *Publisher/Subscriber*

Paulo C. Bartolomeu

Modelo de comunicação *Publisher/Subscriber*



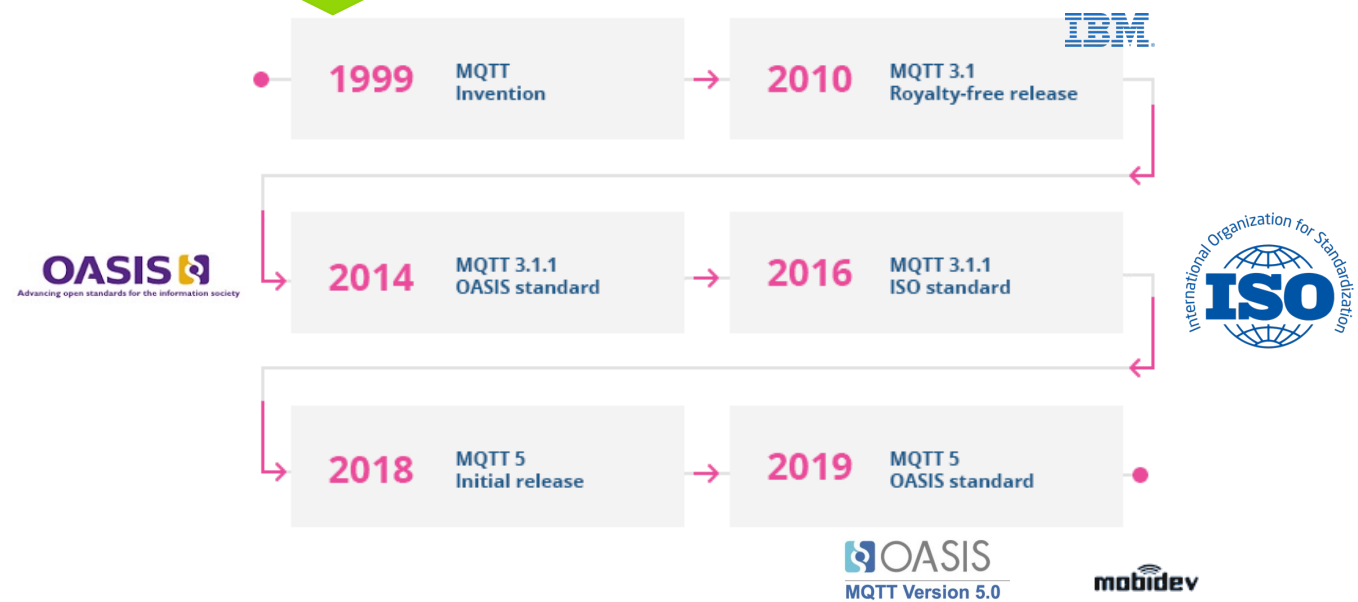
- Mecanismo **simples** e **eficiente** de organizar comunicações em sistemas IoT.
 - os **publishers** enviam dados para um intermediário — o broker
 - os **subscribers** recebem apenas os tópicos a que estão interessados.
 - A separação reduz dependências, facilita a escalabilidade e permite integrar sensores, atuadores e aplicações sem que tenham de conhecer a existência uns dos outros.
- Num contexto onde podem coexistir milhares de dispositivos com recursos limitados e ligações instáveis, este padrão oferece **robustez**, **baixo acoplamento** e um canal comum para distribuição de dados.
- Protocolos como **MQTT** e **AMQP** aplicam esta arquitetura, otimizando o transporte de mensagens conforme os requisitos de fiabilidade, latência ou consumo energético.



Evolução do MQTT

Desenvolvido para permitir que oleodutos e gasodutos comunicassem através de satélites

MQTT 5.0 History



MQTT *Broker* e Clientes

- **MQTT Broker**
 - Servidor central que recebe todas as mensagens publicadas pelos *publishers* e as distribui aos *subscribers* com base nos tópicos.
 - Gere **ligações, autenticação, QoS, retenção de mensagens e sessões persistentes**.
 - Exemplos de **brokers MQTT**:
 - [Eclipse Mosquitto](#): leve, open source, muito usado em ensino e laboratório.
 - [EMQX](#): broker de alta performance, escalável, com dashboards e plugins.
 - [HiveMQ](#) (Community Edition ou Cloud): orientado a produção e integrações empresariais.
 - [RabbitMQ com plugin MQTT](#): broker AMQP popular com suporte adicional para MQTT.
- **Clientes MQTT**
 - Dispositivo ou aplicação que se liga ao *broker* para publicar mensagens (*publisher*), subscrever tópicos (*subscriber*) ou ambos.
 - Pode ser um microcontrolador, um sensor, um serviço na *cloud*, um *script*, uma aplicação móvel ou *web*.
 - Exemplos de **clientes MQTT**:
 - [MQTTX](#): cliente moderno, multiplataforma (GUI e CLI), ideal para testes e depuração de tópicos.
 - [MQTT Explorer](#): ferramenta gráfica para testar publicação e subscrição.
 - [mosquitto pub / mosquitto sub](#): clientes de linha de comandos incluídos no Mosquitto.
 - [Paho MQTT Clients](#): bibliotecas oficiais para Python, C, Java, JavaScript.
 - [ESP32/ESP-IDF MQTT Client](#): biblioteca nativa para dispositivos ESP32.

Modelo de Messaging do MQTT

O modelo de mensagens do MQTT baseia-se em tópicos e subscrições

- Os **tópicos** são *strings* para onde as mensagens são publicadas e às quais os clientes se podem inscrever.
- Os **tópicos** são hierárquicos e podem ter vários níveis separados por barras, tal como um caminho de ficheiro, como por exemplo:

`myhome/kitchen/smartdishwasher`

- As **subscrições** servem para indicar de que tópicos um cliente pretende receber mensagens.
- Quando um **cliente inscreve um tópico** está a informar o *broker* que quer receber as mensagens publicadas nesse tópico.
- O *broker* regista essa subscrição e encaminha para o cliente todas as mensagens que forem publicadas nesse tópico.
- Um **cliente pode inscrever vários tópicos em simultâneo** e que **um tópico pode ter vários subscritores**.

Níveis de QoS para aplicações IoT

- O protocolo MQTT define **três níveis de Qualidade de Serviço (QoS)** que controlam a garantia de entrega das mensagens entre clientes e o *broker*.
- Os níveis de QoS são especialmente relevantes em sistemas *embedded*, onde a estabilidade da rede pode variar e o equilíbrio entre confiabilidade e eficiência é essencial:
 - **QoS 0** - "No máximo uma vez": A mensagem é enviada uma única vez, sem confirmação de receção. É o modo mais rápido e leve, mas não garante que a mensagem chegue ao destino.
 - **QoS 1** - "Pelo menos uma vez": A mensagem é garantidamente entregue, mas pode ser recebida mais de uma vez em caso de retransmissões.
 - **QoS 2** - "Exatamente uma vez": Garante que a mensagem é entregue apenas uma vez, sem duplicação. É o nível mais seguro, mas também o mais pesado em termos de recursos e latência.
- A escolha do **nível de QoS depende do tipo de aplicação**: sensores com atualizações frequentes podem usar QoS 0 para eficiência, enquanto comandos críticos podem exigir QoS 1 ou QoS 2 para maior fiabilidade.
- A **QoS** é definida por mensagem enviada ou por subscrição de um tópico.

Persistência de mensagens para Comunicação IoT fiável

- No MQTT, a **persistência** é conseguida armazenando as mensagens no servidor até serem entregues ao subscritor. O protocolo oferece **três opções principais de persistência**:
 - **Não persistente**
 - Opção predefinida no MQTT. As mensagens não são guardadas no servidor e perdem-se se ocorrer uma falha na rede ou no servidor.
 - Adequado para situações em que as mensagens não são críticas e podem ser facilmente regeneradas.
 - **Persistente com fila (queued persistent)**
 - As mensagens são guardadas no servidor até serem entregues ao subscritor.
 - Se o subscritor estiver indisponível, as mensagens ficam em fila até que este volte a ligar-se.
 - Util quando o subscritor nem sempre está *online* ou quando é necessário garantir que recebe todas as mensagens, mesmo as enviadas durante o período *offline*.
 - **Persistente com confirmação (persistent with acknowledgment)**
 - Oferece o nível mais elevado de fiabilidade.
 - As mensagens são guardadas no servidor até serem entregues e o subscritor tem de confirmar a receção.
 - Se não confirmar, a mensagem é reenviada até ser reconhecida.
 - Indicado para cenários críticos em que é essencial garantir que cada mensagem é recebida e processada.



Sessões MQTT: Persistentes e Clean

- **Sessões persistentes** permitem que um cliente mantenha o seu estado de subscrição e de mensagens ao longo de várias ligações.
 - Quando um **cliente estabelece uma sessão persistente** com um *broker* MQTT, este guarda as informações de subscrição do cliente e quaisquer mensagens ainda não entregues.
 - Assim, se o cliente se desligar e voltar a ligar mais tarde, **consegue retomar a comunicação** de forma contínua.
 - Este mecanismo melhora a QoS, garantindo entrega fiável de mensagens, assegurando níveis de QoS definidos e facilitando reconexões eficientes, mesmo quando há conectividade intermitente.
- Numa **sessão não persistente**, se a ligação for interrompida, o **cliente perde as suas subscrições** e precisa de se voltar a inscrever ao reconectar, o que pode ser penalizante para dispositivos com recursos limitados.
- As sessões persistentes armazenam no *broker* toda a informação relevante do cliente, garantindo que subscrições e mensagens são mantidas mesmo quando o cliente está *offline*. A sessão é identificada pelo **clientId** fornecido pelo cliente durante a ligação.
- Ao estabelecer ligação com o *broker*, o cliente controla a criação ou eliminação de uma sessão persistente através do parâmetro *cleanSession*:
 - **cleanSession = true**
 - O cliente solicita explicitamente uma **sessão não persistente**. Neste caso, se o cliente se desligar, toda a informação e mensagens de uma eventual sessão persistente anterior são descartadas.
 - O cliente começa com uma sessão totalmente nova ao reconectar.
 - **cleanSession = false**
 - O *broker* cria uma **sessão persistente** para o cliente, preservando toda a informação e mensagens mesmo que o cliente fique *offline*.
 - A sessão permanece ativa até o cliente pedir explicitamente uma sessão limpa.
 - Se *cleanSession = false* e já existir uma sessão persistente associada ao mesmo **clientId**, o *broker* reutiliza essa sessão e entrega ao cliente todas as mensagens que estavam em espera no momento da reconexão.



Last Will and Testament (LWT)

- Permite aos clientes especificar uma **mensagem que será publicada automaticamente pelo broker** em seu nome quando ocorrer uma desconexão inesperada (p.ex., perda de conectividade ou baterias descarregadas).
 - Oferece um **mecanismo fiável de comunicação** e garante que os clientes conseguem lidar com falhas de ligação **sem deixar os tópicos num estado inconsistente**.
 - Útil quando os clientes precisam de notificar outros sobre a sua indisponibilidade ou transmitir informação importante em caso de desconexão inesperada.
- Quando um cliente se liga a um *broker*, pode especificar uma **mensagem de “última vontade”**.
 - Esta mensagem segue a estrutura normal de uma mensagem MQTT, incluindo **tópico, indicador de retenção, nível de QoS e payload**.
 - O broker guarda esta mensagem até detetar uma desconexão não graciosa do cliente.
 - Quando isso acontece, o *broker* publica a mensagem de “última vontade” para todos os clientes subscritos no respetivo tópico.
 - Se o cliente se desligar de forma graciosa usando a mensagem DISCONNECT, o *broker* descarta a mensagem LWT armazenada.

Tópicos MQTT

- Um **tópico** é uma **string UTF-8** que serve para **filtrar mensagens** para um cliente ligado. Um tópico é composto por um ou mais níveis separados por uma barra vertical (*forward slash*), que funciona como separador de níveis.



- Exemplos de tópicos MQTT**
 - myhome/groundfloor/livingroom/temperature**: Este tópico representa a temperatura na sala de estar de uma casa situada no rés do chão.
 - USA/California/San Francisco/Silicon Valley**: Esta hierarquia de tópicos pode monitorizar ou trocar informação sobre eventos ou dados relacionados com a região de Silicon Valley, em São Francisco, Califórnia, nos Estados Unidos.
 - Germany/Bavaria/car/2382340923453/latitude**: Esta estrutura de tópicos pode ser utilizada para partilhar as coordenadas de latitude de um determinado automóvel na região da Baviera, na Alemanha.

Boas práticas para uso de tópicos MQTT

- Cada tópico deve **conter pelo menos um caractere**.
- As *strings* dos tópicos **podem incluir espaços** para permitir tópicos mais legíveis ou descritivos.
- Os tópicos são **sensíveis a maiúsculas e minúsculas**; por isso, “myhome/temperature” e “MyHome/Temperature” são considerados dois tópicos diferentes.
- A **barra “/” isolada é um tópico válido** e pode ser usada para representar um tópico amplo ou funcionar como um *wildcard* para subscrever vários tópicos em simultâneo.

Tópicos MQTT: *wildcards*

- **MQTT wild-card de nível único: +**

- O *wildcard* de um único nível é representado pelo símbolo + e permite substituir um único nível do tópico.
- Ao subscrever um tópico que inclua este *wildcard*, qualquer tópico que contenha uma *string* arbitrária nesse nível será correspondido.

single-level
wildcard
↓
myhome / groundfloor / + / temperature
|
only one level

- Por exemplo, uma subscrição a `myhome/groundfloor/+/temperature` pode produzir o seguinte resultado:

- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / brightness
- ✗ myhome / firstfloor / kitchen / temperature
- ✗ myhome / groundfloor / kitchen / fridge / temperature

Tópicos MQTT: *wildcards*

- **MQTT wild-card multi-nível: #**

- O *wildcard* de múltiplos níveis abrange vários níveis de tópicos. É representado pelo símbolo # e deve ser colocado como o último caractere do tópico, sendo precedido por uma barra (/).

multi-level
wildcard
↓
myhome / groundfloor / #

only at the end
multiple topic levels

- Quando um cliente subscreve um tópico com este *wildcard*, recebe todas as mensagens cujo tópico começa pelo padrão que antecede o símbolo #, independentemente da profundidade ou quantidade de níveis seguintes. Se o tópico for simplesmente “#”, o cliente recebe todas as mensagens enviadas ao broker MQTT.

- ✓ myhome / groundfloor / livingroom / temperature
- ✓ myhome / groundfloor / kitchen / temperature
- ✓ myhome / groundfloor / kitchen / brightness
- ✗ myhome / **firstfloor** / kitchen / temperature

Advanced Message Queuing Protocol (AMQP)

- **Protocolo de mensagens orientado a filas**, usado sobretudo em sistemas distribuídos e aplicações de grande escala.
- É mais **pesado** e mais **completo** do que o MQTT, oferecendo controlo detalhado sobre filas, trocas de mensagens, roteamento e fiabilidade.

Elementos fundamentais do AMQP

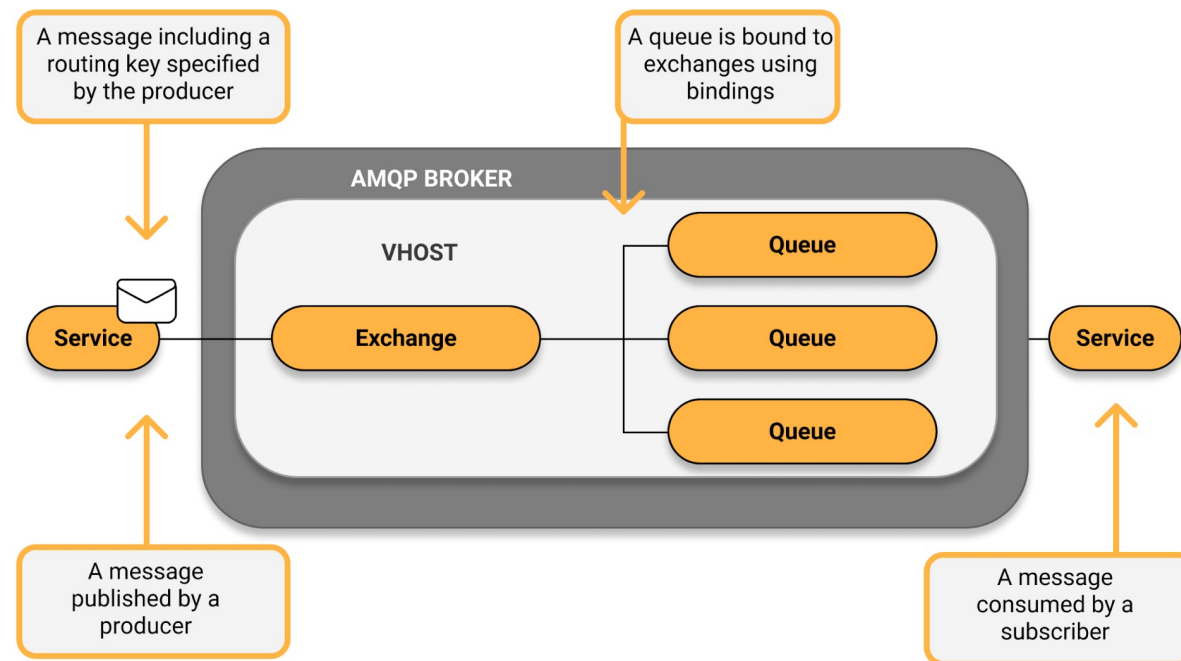
- **Broker AMQP**: O servidor responsável por armazenar, encaminhar e distribuir mensagens. Exemplos típicos: [RabbitMQ](#), [ActiveMQ](#).
- **Produtores e Consumidores** enviam mensagens para o *broker* e recebem mensagens a partir das filas, respetivamente.
- **Exchanges**: Em AMQP, o produtor não envia mensagens diretamente para uma fila. Envia-as para uma *exchange*, que decide a que filas a mensagem deve ser encaminhada. O *exchange* pode ser:
 - Durável: persiste até ser apagada.
 - Temporário: desaparece quando o servidor encerra.
 - Auto-deleted: removido quando deixa de ser utilizado.

Um *exchange* pode implementar diferentes algoritmos de encaminhamento. Cada mensagem é encaminhada para as filas que correspondem às regras configuradas (bindings).

- **Queues (filas)** são buffers onde as mensagens ficam armazenadas até serem consumidas. Podem ser persistentes (sobrevivem a reinícios) ou voláteis.
- **Bindings** são associações entre uma *exchange* e uma fila. Definem as regras que determinam que mensagens chegam a que fila.

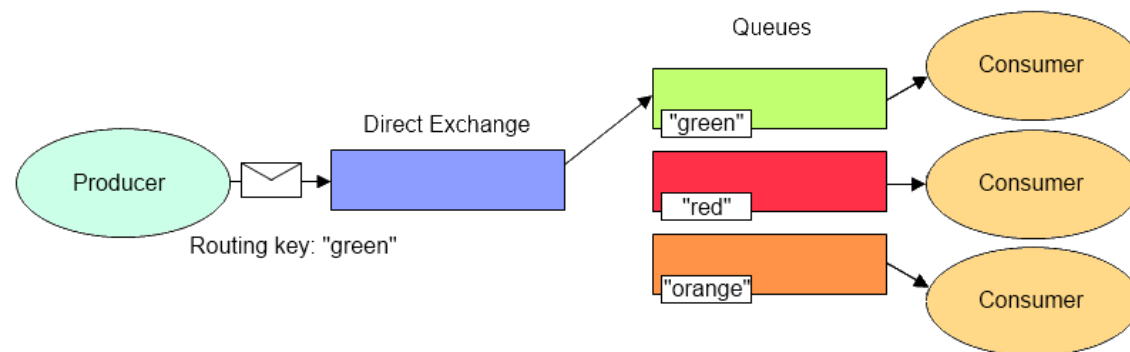


Advanced Message Queuing Protocol (AMQP)



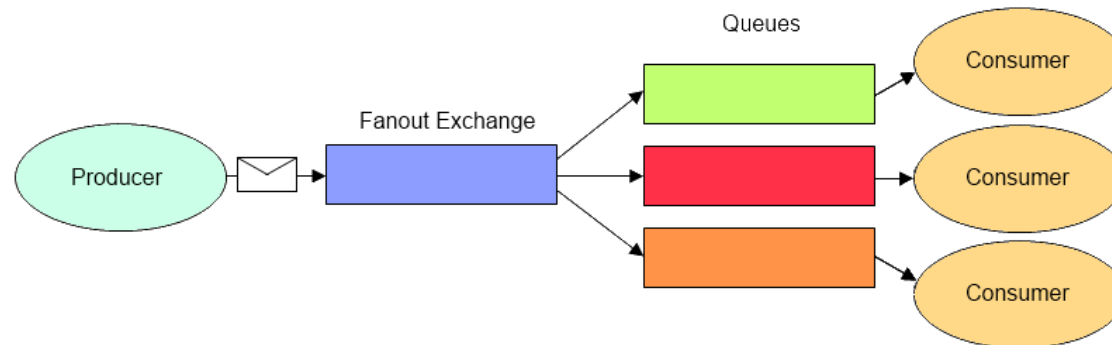
AMQP: Tipos de Exchange

- O tipo **direct exchange** é o mais direto: uma mensagem só é encaminhada quando a sua *routing key* coincide exatamente com a *binding key* definida pela fila.
- Esta correspondência absoluta torna o **comportamento previsível** e adequado quando existe um mapeamento fixo entre tipos de mensagens e destinos.
- É uma **solução utilizada quando cada mensagem tem um destinatário claro**, como num sistema de comandos individuais ou de encaminhamento rígido.



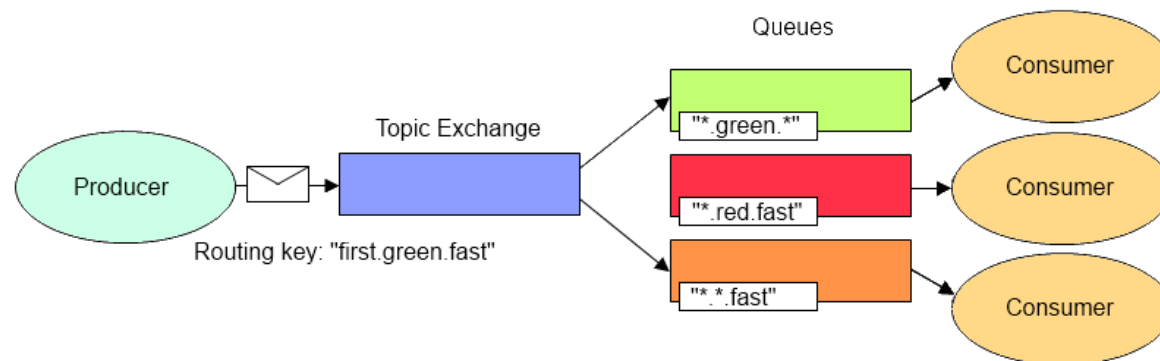
AMQP: Tipos de Exchange

- O tipo **fanout exchange** representa o extremo oposto. Não olha para chaves nem padrões e limita-se a difundir todas as mensagens para todas as filas ligadas.
- Funciona como um mecanismo de **broadcast**, ideal para cenários em que vários sistemas precisam de receber as mesmas mensagens, por exemplo para replicação de dados, alertas generalizados ou registos de eventos que devam ser processados por múltiplos serviços em paralelo.



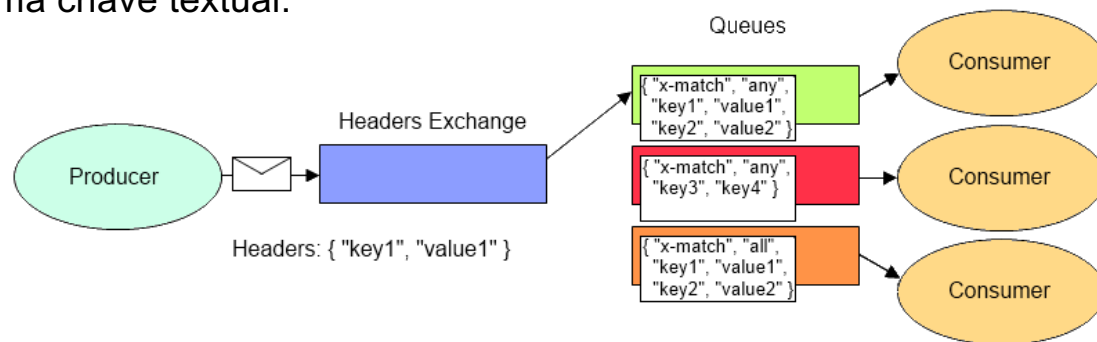
AMQP: Tipos de Exchange

- O tipo **topic exchange** introduz flexibilidade ao permitir que a *routing key* seja tratada como uma expressão hierárquica sujeita a padrões.
- Os **wildcards** permitem corresponder vários tópicos de forma inteligente, aproximando-se do modelo de tópicos do MQTT.
- Esta abordagem é importante quando se lida com dados organizados por categorias, localizações ou tipos de eventos, permitindo que **subscritores recebam apenas o subconjunto de mensagens relevante para o seu domínio**.



AMQP: Tipos de Exchange

- O **headers exchange** afasta-se da lógica baseada em chaves e passa a encaminhar mensagens com base nos cabeçalhos que estas transportam.
- Cada fila define os critérios que devem estar presentes nos headers para receber a mensagem, podendo exigir coincidência total ou parcial.
- Este tipo de **exchange** é particularmente útil em contextos empresariais mais exigentes, onde a decisão depende de atributos específicos de cada mensagem, muitas vezes combinando vários parâmetros em simultâneo.
- É o mais flexível e o mais pesado, mas também o mais expressivo quando as regras de distribuição não podem ser descritas apenas por uma chave textual.



MQTT vs. AMQP

Pros of AMQP:	Pros of MQTT:
Has built-in integrations for TLS and SASL for greater security	Is lightweight and battery-friendly
Supports many different messaging patterns	Offers 3 QoS levels to support reliable messaging
Offers three levels of QoS for message deliverability and reliability	Works well over unreliable networks
Can be extended very easily	Allows for security through TLS
Has many open-source libraries to make implementation easy	Has many open-source libraries to ease development and implementation
	Low bandwidth usage
Cons of AMQP:	Cons of MQTT:
Requires higher bandwidth, compared to MQTT	Only supports the publish-subscribe messaging pattern
Has bigger messages, which means slower transmission speeds	Not as easily extensible, compared to AMQP
	Less built-in security features, compared to AMQP

cometchat

Kahoot! Time

